

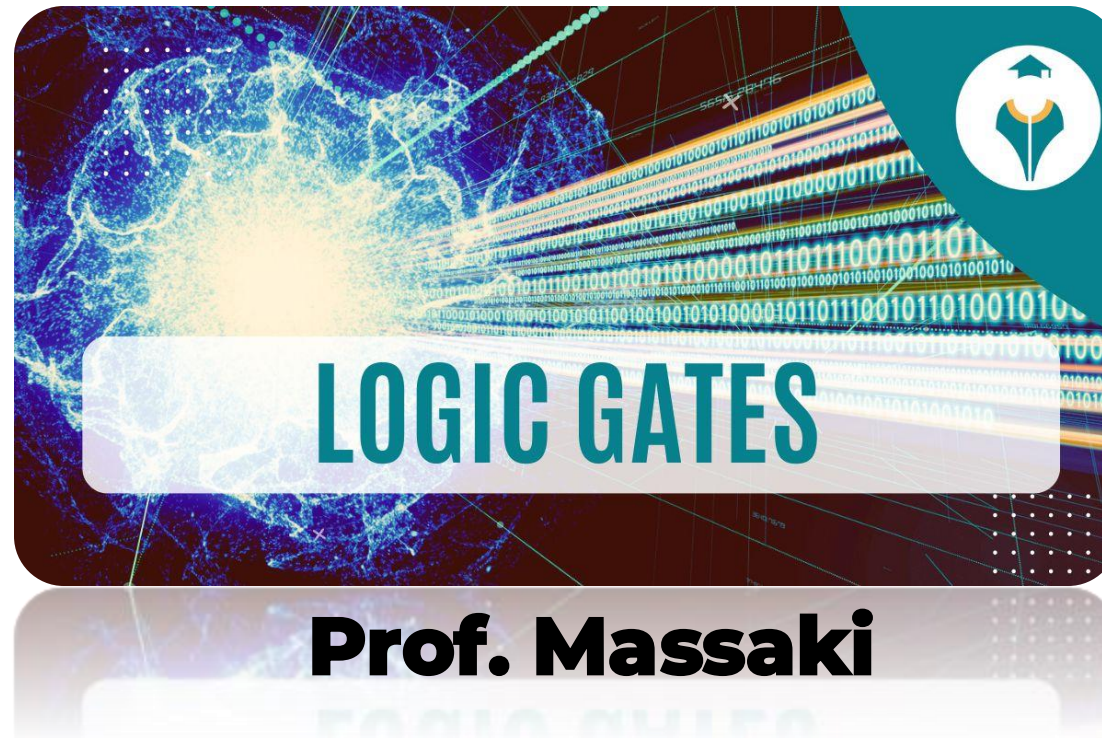
SISTEMAS ELETRÔNICOS DIGITAIS E SISTEMAS EMBARCADOS

UM APRENDIZADO ATUAL E EFICAZ

Prof. Massaki de O. Igarashi

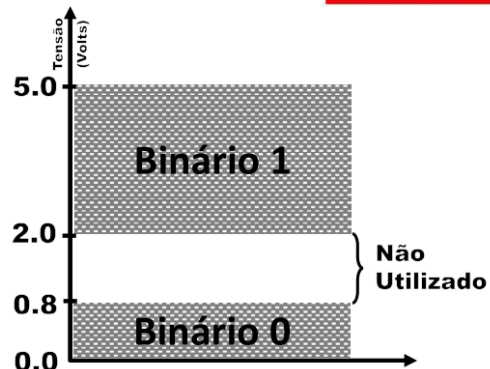
AULA 07

Portas lógicas e Álgebra Booleana com Microntroladores



INTRODUÇÃO

Sistemas embarcados em seu interior não sabem interpretar números decimais (0, 1, 2, ...) porque essencialmente funcionam sob **lógica binária** (0 = desligado/falso ou 1 = ligado/verdadeiro). Mas esta lógica binária pode ser aplicável em relés e transistores de junção bipolar. Estes componentes, comuns da eletrônica, se comportam como um **circuito digital**, pois entregam a característica de mudanças de estados, representados por variações de níveis de tensões, no qual os intervalos são de 0-0,8 Volts para falso e 2-5 Volts para verdadeiro.



6

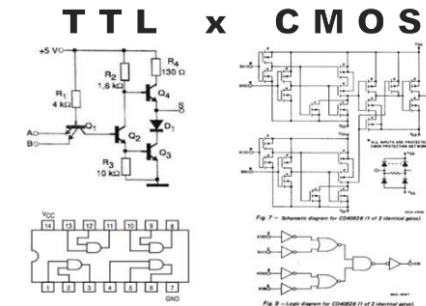
INTRODUÇÃO

TTL (Transistor-Transistor Logic) e CMOS (Complementary Metal-Oxide-Semiconductor) são duas famílias de tecnologias de circuitos digitais que diferem significativamente em relação a níveis lógicos e valores de tensão.

TTL utiliza transistores bipolares e opera com uma **tensão de alimentação típica de 5V**. Em TTL, um nível lógico "0" (baixo) é representado por uma tensão entre 0V e 0.8V, enquanto um nível lógico "1" (alto) varia entre 2V e 5V. Por outro lado, CMOS usa transistores de efeito de campo e opera em uma faixa mais ampla de tensões, geralmente variando de 3V a 15V, com níveis lógicos "0" e "1" definidos em relação à tensão de alimentação.

Em CMOS, um nível lógico "0" é geralmente abaixo de 1/3 da tensão de alimentação, e um nível lógico "1" é acima de 2/3 da tensão de alimentação.

Em resumo, TTL tende a ter uma faixa de tensão mais estreita e valores mais fixos, enquanto CMOS oferece maior flexibilidade e eficiência energética com faixas de tensão mais amplas.



7

Nas últimas aulas aprendemos sobre portas lógicas!

Porta Lógica E (AND)



A **porta E (AND)** pode ser vista como **dois interruptores** conectados **em série**. Conforme a figura ao lado esquerdo:

se ambos os interruptores 'a' e 'b' estiverem **OFF** (também chamados de 0), a **lâmpada** (o círculo com a cruz) **não acenderá**;

ela **também** está **OFF** (= 0). Se o interruptor 'a' estiver **ON** (=1) e o interruptor 'b' estiver **OFF** (=0), a lâmpada ainda estará **OFF**.

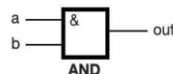
A **lâmpada** (o círculo com a cruz) **só acenderá** se ambos os interruptores 'a' e 'b' estiverem **ON** (ou seja, em nível lógico 1);

Essas possibilidades são demonstradas na tabela verdade.

À direita o símbolo europeu de uma porta AND.



a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1



8

Porta Lógica OU (OR)



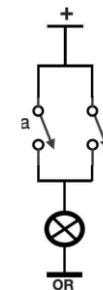
A **porta OU (OR)** é uma combinação de **duas chaves** conectadas **em paralelo**.

Basta apenas **uma das chaves**, 'a' ou 'b', na **posição ON** (=1) para que o resultado seja **ON** (=1).

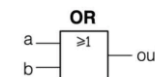
A **lâmpada só apagará** se ambos os interruptores 'a' e 'b' estiverem **OFF** (também chamados de 0);

Essas possibilidades são demonstradas na tabela verdade.

À direita o símbolo europeu de uma porta AND.



a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1



10

Porta Lógica

OU exclusivo (XOR)

A porta 'OU Exclusiva' (XOR) consiste em dois interruptores, ambos com dois polos ligados em série. A configuração destes dois interruptores é que ambos na mesma configuração permitem você ligar ou desligar a luz. Este circuito é frequentemente aplicado em salas onde você pode ligar a luz na porta e desligar a mesma luz do outro lado da sala (popularmente conhecido como ligação paralela).

Ao verificar na figura ao lado, a luz só ligará quando ambos os interruptores 'a' e 'b' estiverem na posição oposta. No caso da imagem ao lado, os interruptores estão conectados na mesma posição (mas de cabeça para baixo) - portanto, o resultado é OFF (=0).

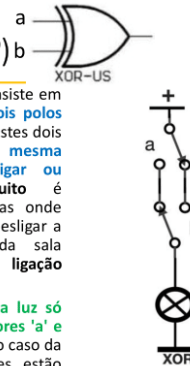
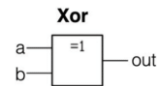


Tabela Verdade

a	b	XOR
0	0	0
0	1	1
1	0	1
1	1	0



SENAI

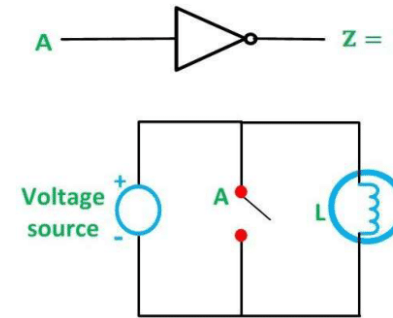
12

<https://circuitglobe.com/not-gate.html>

Porta Lógica

NÃO (NOT)

A porta NÃO (NOT), também é conhecida como Inverter, é uma porta de entrada única e saída única. É porque ela realiza a inversão do sinal binário aplicado, ou seja, converte 0 em 1 ou 1 em 0. Em outras palavras, o gate tem um sinal de entrada alto somente quando seu sinal de entrada é baixo, por isso esse tipo de porta é conhecida como not gate. O símbolo da porta NOT é um triângulo com um círculo em sua extremidade. Este círculo é conhecido como círculo de inversão. Ela fornece o complemento do sinal de entrada. Também é conhecida como dispositivos de tomada de decisão porque tem apenas uma entrada. O diagrama do circuito de comutação da porta NOT é mostrado



SENAI

13

Nas últimas aulas aprendemos sobre portas lógicas!

PORTAS LÓGICAS Símbolos e CIs

SENAI

Revisões Módulo 2 Álgebra e Lógica Booleana									
Portas elementares									
Diagrama do Chip	U ₁ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₂ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₃ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₄ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₅ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₆ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₇ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₈ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂	U ₉ A ₁ B ₁ C ₁ A ₂ B ₂ C ₂
CHIP TTL	7408	7432	7407	7404	7400	7402	7486	74266	
Desenho das portas									
CHIP CMOS	4073	4071	4049	4093	4001	4070	74HC266		
Saída	AND A ₁ B ₁	OR A ₁ + B ₁	Buffer A	NOT A ₁	NAND A ₁ B ₁	NOR A ₁ + B ₁	XOR A ₁ B ₁	XNOR A ₁ B ₁	
Posição	B	A	Multiplicação	Soma	Inverte	Nega saída	Nega saída	Exclusiva	Coincidência
0	0	0	0	0	0	1	1	0	1
1	0	1	0	1	1	0	0	1	0
2	1	0	0	1	0	1	0	1	0
3	1	1	1	1	0	0	0	0	1

Fonte: <https://nicmania.net/tic3/rec/sd/indexsdac10pasta/img/portasLogicas/portasLogicasresumo.JPG>

23/03/2025 17:24:31 14

PORTAS LÓGICAS Circuitos e detalhes

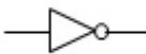
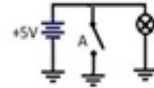
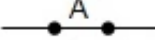
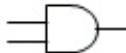
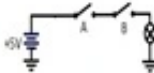
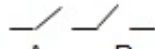
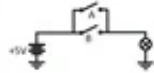
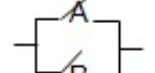
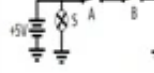
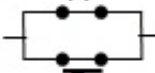
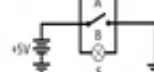
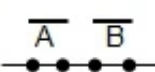
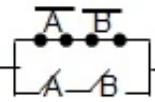
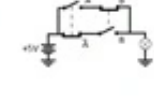
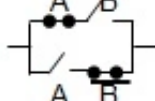
SENAI

Operador Proposicional	Operado Booleano	Compuerta	Circuito	Lógica Cableada	Tabela de verdad															
~	$\bar{A}$				<table><tr><td>A</td><td>0</td><td>1</td></tr><tr><td>$\bar{A}$</td><td>1</td><td>0</td></tr></table>	A	0	1	$\bar{A}$	1	0									
A	0	1																		
$\bar{A}$	1	0																		
^	$A \bullet B$				<table><tr><td>A</td><td>B</td><td>$A \bullet B$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$A \bullet B$	0	0	0	0	1	0	1	0	0	1	1	1
A	B	$A \bullet B$																		
0	0	0																		
0	1	0																		
1	0	0																		
1	1	1																		
v	$A + B$				<table><tr><td>A</td><td>B</td><td>$A + B$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$A + B$	0	0	0	0	1	1	1	0	1	1	1	1
A	B	$A + B$																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	1																		
$\bar{\wedge}$	$\overline{A \bullet B}$				<table><tr><td>A</td><td>B</td><td>$\overline{A \bullet B}$</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$\overline{A \bullet B}$	0	0	1	0	1	1	1	0	1	1	1	0
A	B	$\overline{A \bullet B}$																		
0	0	1																		
0	1	1																		
1	0	1																		
1	1	0																		
$\bar{v}$	$\overline{A + B}$				<table><tr><td>A</td><td>B</td><td>$\overline{A + B}$</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$\overline{A + B}$	0	0	1	0	1	0	1	0	0	1	1	0
A	B	$\overline{A + B}$																		
0	0	1																		
0	1	0																		
1	0	0																		
1	1	0																		
$\leftrightarrow$	$A \odot B$				<table><tr><td>A</td><td>B</td><td>$A \odot B$</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$A \odot B$	0	0	1	0	1	0	1	0	0	1	1	1
A	B	$A \odot B$																		
0	0	1																		
0	1	0																		
1	0	0																		
1	1	1																		
$\nleftrightarrow$	$A \oplus B$				<table><tr><td>A</td><td>B</td><td>$A \oplus B$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$A \oplus B$	0	0	0	0	1	1	1	0	1	1	1	0
A	B	$A \oplus B$																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	0																		

<https://matematicadiscretasblog.wordpress.com/compuertas-y-circuitos-logicos/>

27

Portas Lógicas: Circuitos e Detalhes

Operador Proposicional	Operado Booleano	Compuerta	Circuito	Lógica Cableada	Tabla de verdad															
$\sim$	$\overline{A}$				<table><tr><td>A</td><td>0</td><td>1</td></tr><tr><td>$\overline{A}$</td><td>1</td><td>0</td></tr></table>	A	0	1	$\overline{A}$	1	0									
A	0	1																		
$\overline{A}$	1	0																		
$\wedge$	$A \bullet B$				<table><tr><td>A</td><td>B</td><td>$\bullet$</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	A	B	$\bullet$	0	1	0	0	0	0	1	1	1	1	0	1
A	B	$\bullet$																		
0	1	0																		
0	0	0																		
1	1	1																		
1	0	1																		
$\vee$	$A + B$				<table><tr><td>A</td><td>B</td><td>$+$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$+$	0	0	0	0	1	1	1	0	1	1	1	1
A	B	$+$																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	1																		
$\neg$	$\overline{A \bullet B}$				<table><tr><td>A</td><td>B</td><td>$\neg$</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$\neg$	0	0	1	0	1	1	1	0	1	1	1	0
A	B	$\neg$																		
0	0	1																		
0	1	1																		
1	0	1																		
1	1	0																		
$\underline{\vee}$	$\overline{A + B}$				<table><tr><td>A</td><td>B</td><td>$\overline{+}$</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$\overline{+}$	0	0	1	0	1	1	1	0	1	1	1	0
A	B	$\overline{+}$																		
0	0	1																		
0	1	1																		
1	0	1																		
1	1	0																		
$\longleftrightarrow$	$A \odot B$				<table><tr><td>A</td><td>B</td><td>$\odot$</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$\odot$	0	0	1	0	1	0	1	0	0	1	1	1
A	B	$\odot$																		
0	0	1																		
0	1	0																		
1	0	0																		
1	1	1																		
$\nleftrightarrow$	$A \oplus B$				<table><tr><td>A</td><td>B</td><td>$\oplus$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$\oplus$	0	0	0	0	1	1	1	0	1	1	1	0
A	B	$\oplus$																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	0																		

Portas Lógicas: Símbolos e CIs

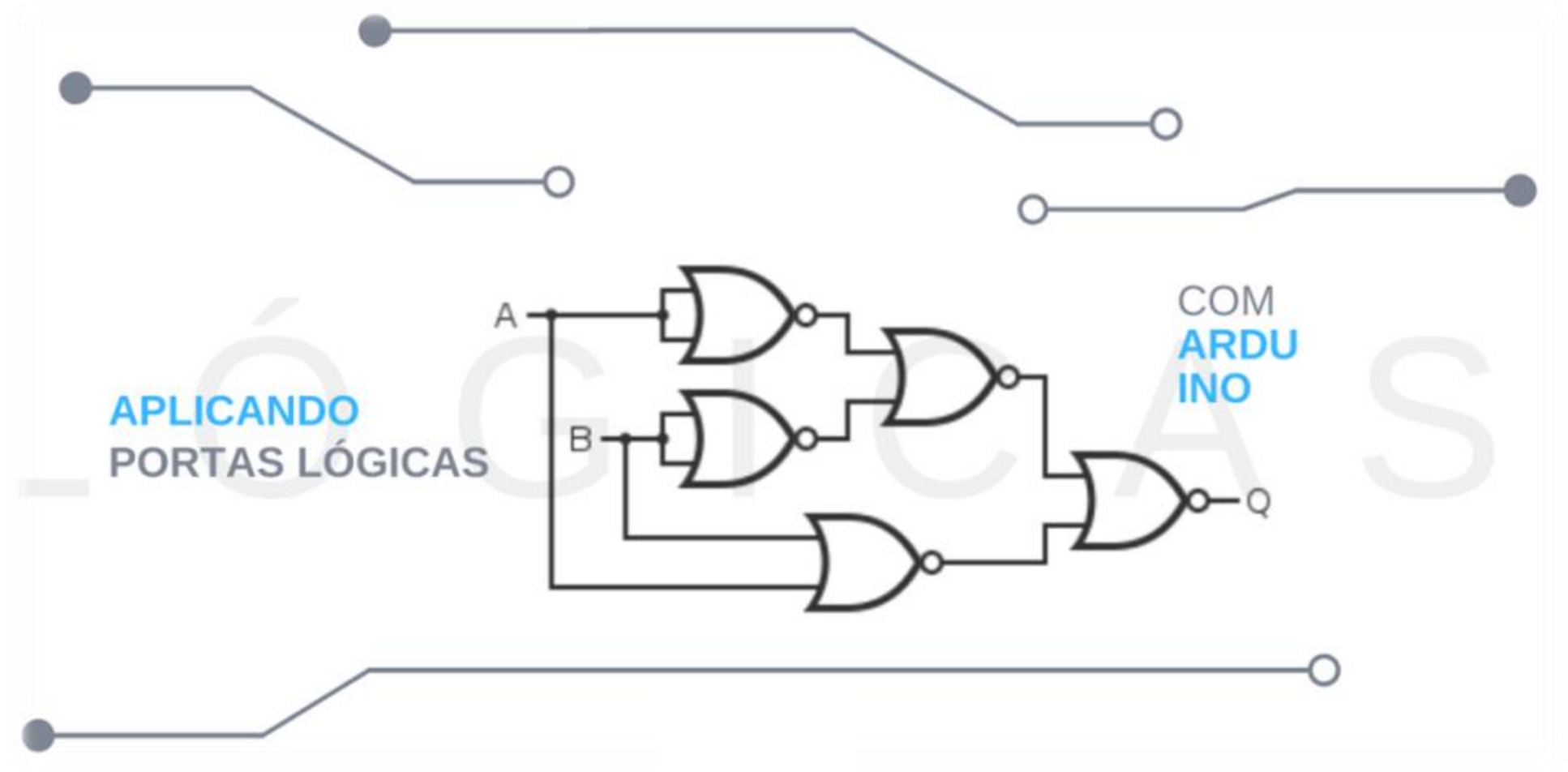
Revisões Módulo 2 Álgebra e Lógica Booleana

Diagrama do Chip	Portas elementares			7404 Hex Inverters	7400 Quad 2-input NAND Gates	7402 Quad 2-input NOR Gates	7486 Quad 2-input ExOR Gates	74266
CHIP TTL	7408	7432	7407	7404	7400	7402	7486	74266
Desenho das portas								
CHIP CMOS	4073	4071	4049		4093	4001	4070	74HC266
	AND	OR	Buffer	NOT	NAND	NOR	$A \cdot \bar{B} + \bar{A} \cdot B$	$A \cdot B + \bar{A} \cdot \bar{B}$
	$A \cdot B$	$A + B$	A	$\bar{A}$	$\overline{A \cdot B}$	$\overline{A + B}$	XOR	XNOR
Saída	$A \cdot B$	$A + B$	A	$\bar{A}$	$\overline{A \cdot B}$	$\overline{A + B}$	$A \oplus B$	$\overline{A \oplus B}$
Posição	B A	B A	B A	B A	B A	B A	B A	B A
0	0 0	0 0	0 0	1 1	1 1	1 1	0 0	1 1
1	0 1	1 0	1 0	0 0	1 1	0 0	1 1	0 0
2	1 0	0 1	0 1	1 0	0 1	1 0	0 1	0 1
3	1 1	1 1	1 1	0 0	0 0	0 0	0 0	1 1

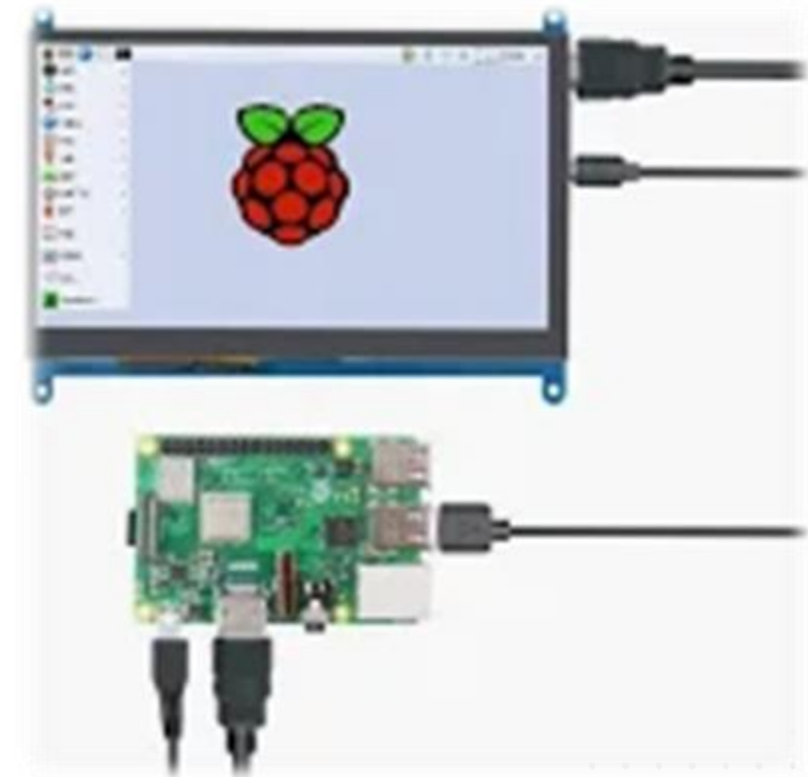
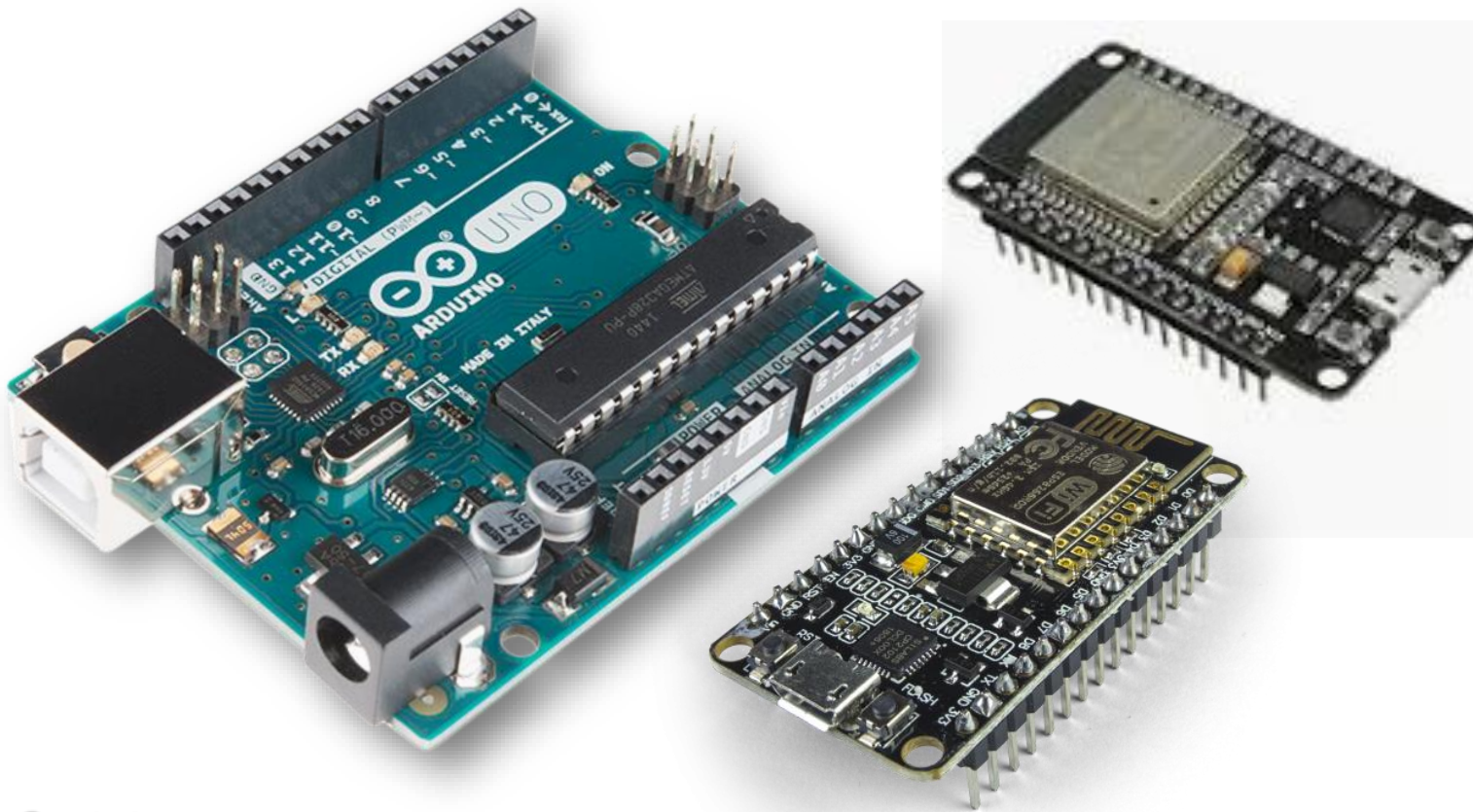
Fonte: <https://ticmania.net/tic3/rec/sd/indexsdac10pasta/img/portasLogicas/portasLogicasresumo.JPG>

PORTAS LÓGICAS

e a álgebra booleana



Primeiros passos com SISTEMAS EMBARCADOS



Contato:
massaki.igarashi@senai.edu.br

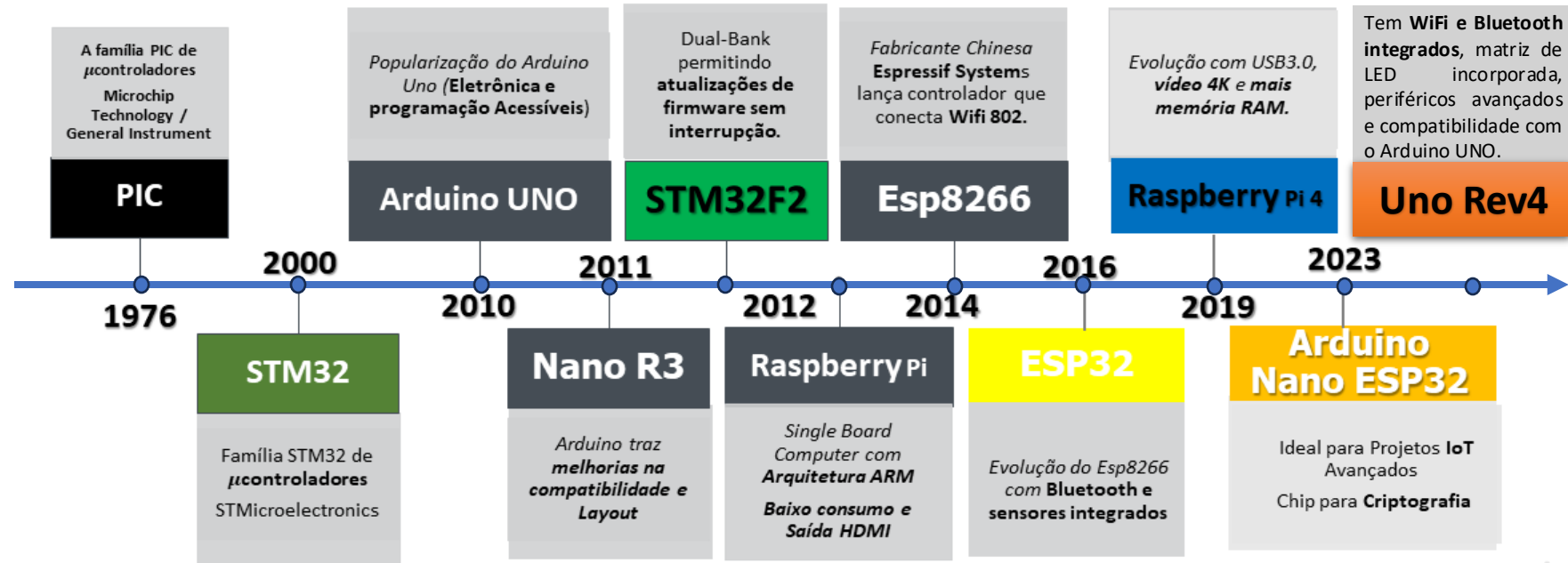
MICROCONTROLADORES

Linha do Tempo

Os **microcontroladores** são compactos, com recursos integrados e são usados em aplicações embarcadas, enquanto **microprocessadores** são mais poderosos e usados em computadores e sistemas complexos. Suas diferenças incluem capacidade de processamento, memória, portas de E/S e consumo de energia elétrica.

Aplicações: controlar máquinas, processar dados e implementar comunicação e interfaces de usuário, possibilitando a eficiência e inovação nos sistemas industriais. Compreender suas funções, diferenças e aplicações é fundamental para quem trabalha nesta área.

Linha do Tempo dos uControladores usados para fins didáticos-profissionais



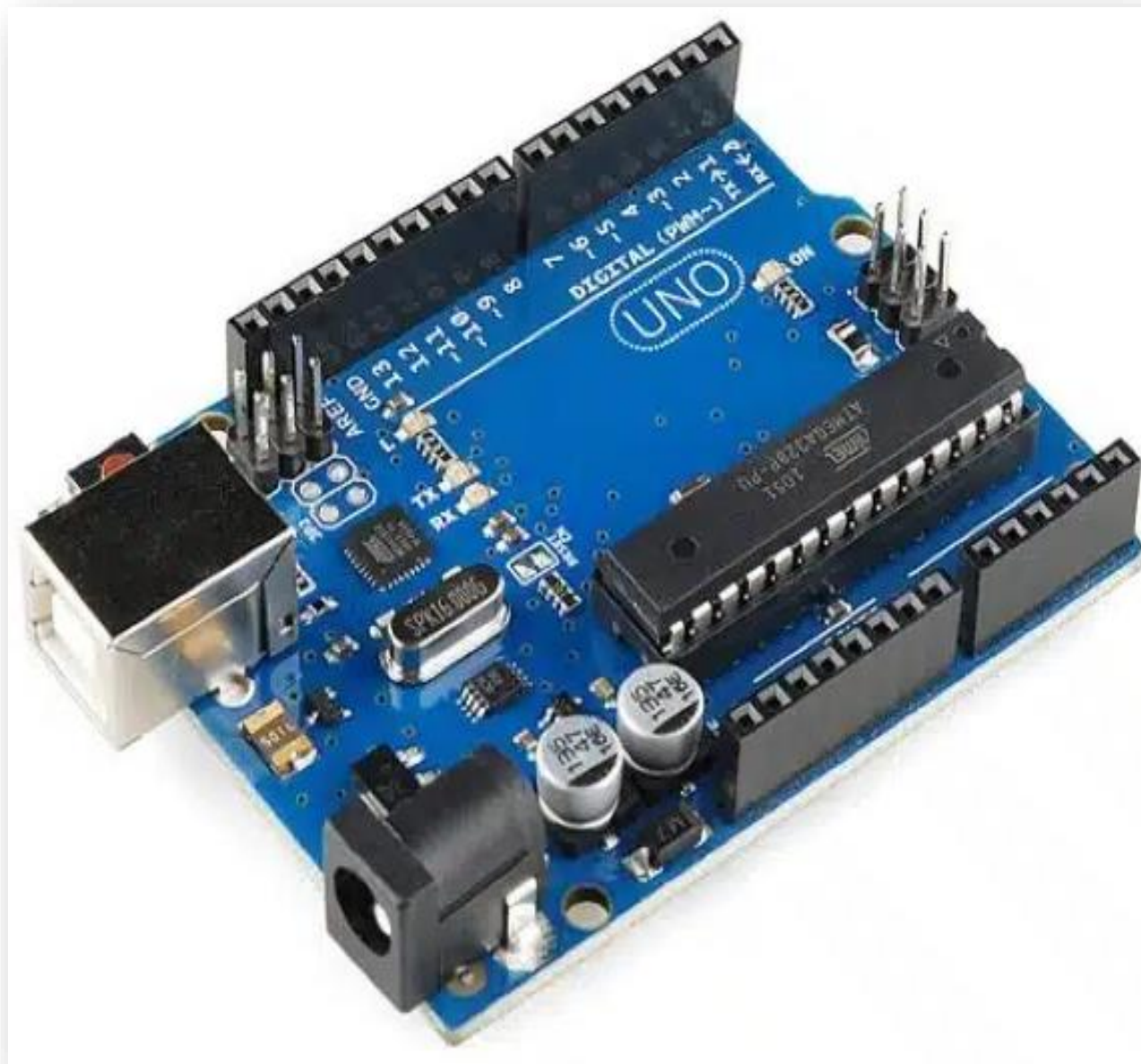
Um **sistema embarcado** é um sistema de computação dedicado a realizar uma ou mais funções específicas. É integrado ao hardware do qual faz parte e é projetado para operar de forma independente ou como parte de um sistema maior. Principais características: **especialização** (projetados para executar tarefas específicas); operam em **Tempo Real**; apresentam **Confiabilidade**; **Eficiência Energética** (menor consumo de energia); **Tamanho Compacto** e **Baixo Custo**;

O MICROCONTROLADOR

Arduino Uno

Microcontrolador	ATmega328P
Tensão de Operação	5 Volts
Tensão de Entrada (Recomendada)	7-12V
Tensão de Entrada (Limites)	6-20V
Pinos I/O	14 (6 com PWM)
Pinos Analógicos	6
Corrente por pino I/O	40 mA
Corrente no pino 3.3v	50 mA
Memória Flash	32 kB
SRAM	2 kB
EEPROM	1 kB
Velocidade de Clock	16 MHz

Tabela 1 - Especificações do *Arduino UNO*



Entendendo como se estrutura o Programa C++ do Arduino Uno

The image shows the Arduino IDE interface with several callouts explaining the structure of a C++ program for an Arduino Uno. The main window displays a sketch named 'sketch_aug28a.ino' with the following code:

```
1 void setup() {  
2   // put your setup code here, to run once:  
3  
4 }  
5  
6 void loop() {  
7   // put your main code here, to run repeatedly:  
8  
9 }  
10
```

Callouts include:

- Botão Upload:** Ao clicar no botão upload você **compilará** seu código e em **seguida** gravará o mesmo na **memória flash** do microcontrolador da sua placa.
- Função setup():** Aqui você insere as **configurações iniciais** que **executarão uma única vez**, ao ligar seu Arduino ou ao pressionar o botão reset da placa.
- Função loop():** Na função loop você insere **tudo o que precisará ser executado e repetido a cada ciclo de clock** do microcontrolador da placa.

Other elements in the image include a speech bubble saying 'Eles crescem tão rápido...' (They grow so fast...), a small image of an Arduino Uno board, and a 'Click to expand' button.



ATENÇÃO: Antes de sair gravando seu microcontrolador, teste seu código em plataformas online como



AUTODESK
TINKERCAD

e/ou



Exemplos de configurações usadas na função Loop

Os símbolos `{` e `}` servem, respectivamente, p/ informar o **início** e o **fim** de uma função ou para especificar diferentes ações a serem executadas dentro de uma estrutura de controle de fluxo (condicional ou loop repetição).;

```
void setup() {  
  
  Serial.begin(9600); // Inicializa Comunicação Serial a 9600 bps  
  
  lcd.begin(16, 2); // Inicializa display LCD (16 colunas x 2 linhas)  
  
  pinMode(10, INPUT); // Configura Pino Digital D10 p/ entrada digital (Botão)  
  
  pinMode(13, OUTPUT); // Configura Pino Digital D13 como saída digital  
  
}
```



Função **setup()**

Aqui você insere as **configurações iniciais** que **executarão uma única vez**, ao ligar seu Arduino.

Função **loop()**

Na função loop você insere **tudo o que precisará ser executado e repetido a cada ciclo de clock** do microcontrolador da placa.

`/*` Já os símbolos de **`/*`** e **`*/`** são usados, respectivamente, no início e no final de um comentário de várias linhas ou, caso sea desejado apenas um comentário único, ou seja, uma explicação junto à seu comando de execução, basta usar **`//`** duas barras de divisão consecutivas no início do comentário.**`*/`**

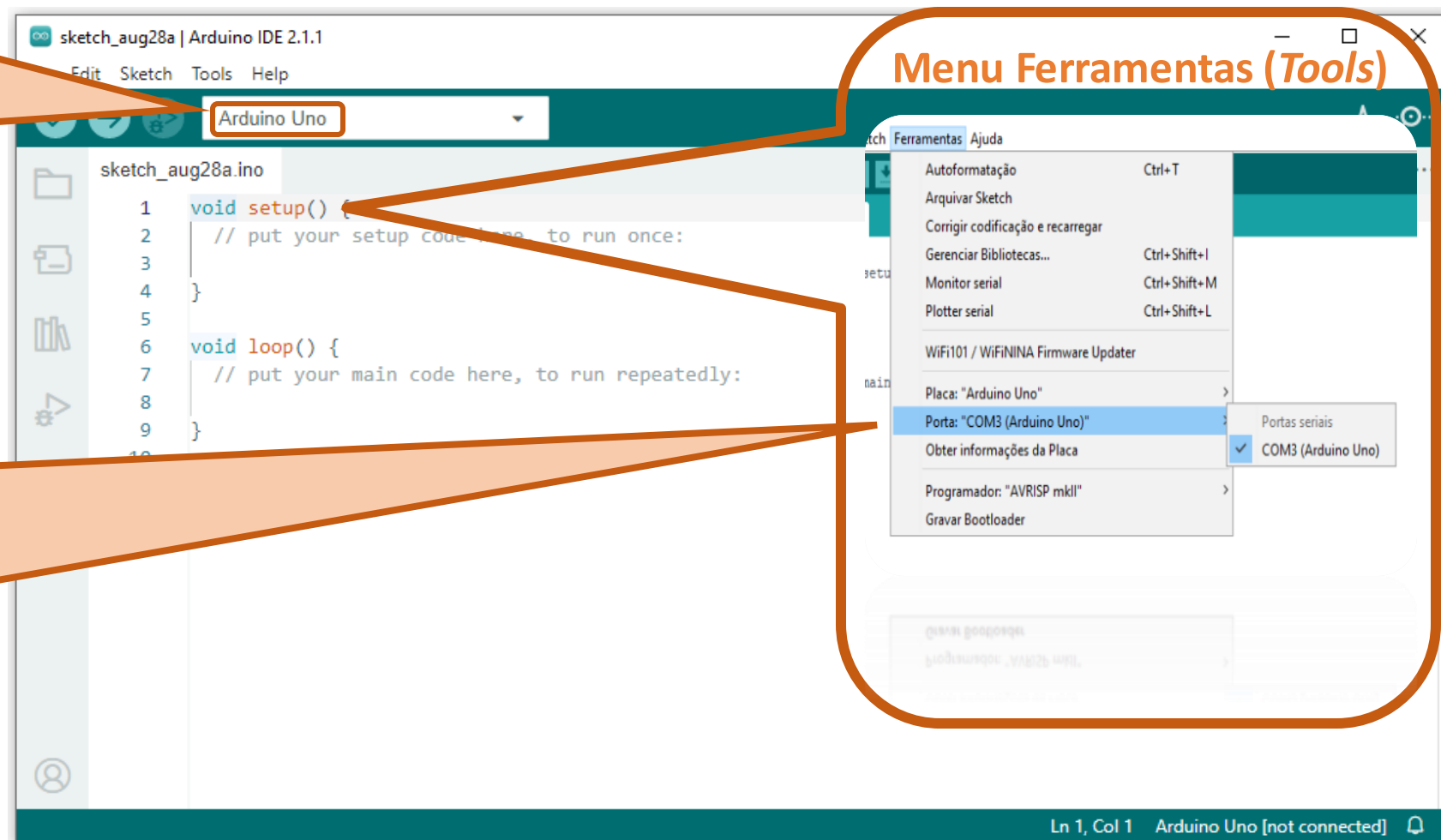
DICA Nº 1

1ª) Selecione a Placa correta!

Preste atenção para escolher a placa correta para que não ocorram erros de gravação no seu kit de microcontrolador; isto pode ocasionar queima da memória flash do seu microcontrolador.c

2ª) Selecione a Porta Serial!

Praticamente todas as placas didáticas que existem no mercado se comunicará com seu computador e serão gravadas por meio da porta serial de dados; portanto, é importante que você preste muita atenção para selecionar a porta correta que será usada por seu microcontrolador.

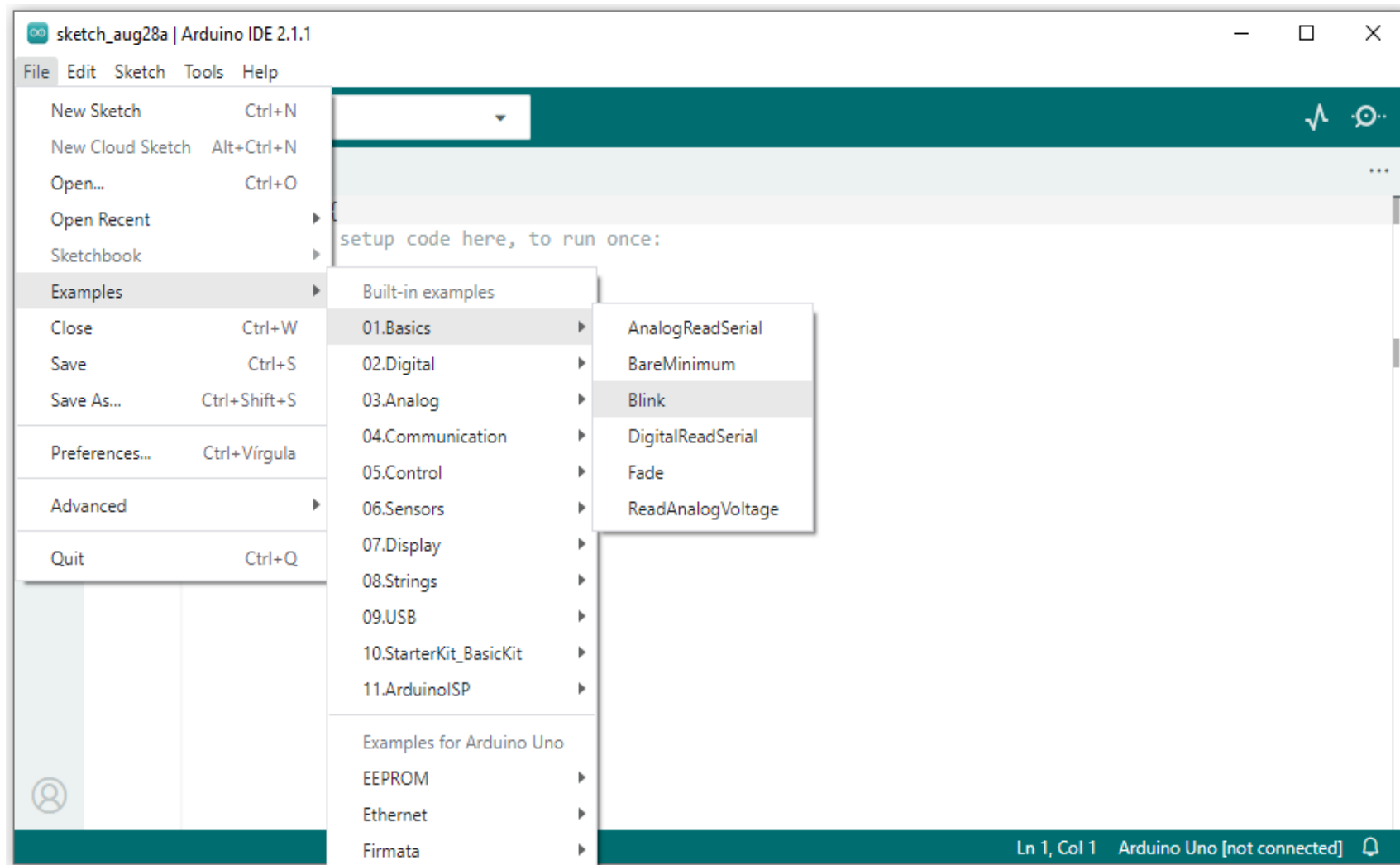


DICA Nº 2

Sempre testar a sua placa antes de embarcar qualquer novo código: para isso programe o código do exemplo *Blink* que se encontra no *Menu*

File > Examples > 01. Basics > Blink

Isto poupará sua dor de cabeça e tempo caso a placa tenha algum problema de fabricação; independente do seu código ou qualquer erro na lógica do programa.



DICA Nº 3



ATENÇÃO

Nunca se esqueça de ir no **Menu Ferramentas (*Tools*) > Gerenciador de bibliotecas (*Manage Libraries*)** e instale as bibliotecas necessárias para que todos os periféricos adicionados por você ao seu projeto funcionem adequadamente.



01 - INTRODUÇÃO



01.1 – Variáveis

Na Linguagem de programação



Em programação, um **identificador** é um nome atribuído a uma **variável** de memória, uma estrutura de dados, uma classe, um objeto, um procedimento, uma função, um comando ou palavra reservada da linguagem.

Variável é tudo que está **sujeito a variações**, que é **incerto, instável ou inconstante**. Quando se fala de **computadores**, é preciso ter em mente que o volume de dados a serem tratados é grande e diversificado. Dessa forma, os **dados a serem processados são bastante variáveis**.

Todo dado a ser armazenado na memória de um computador deve ser previamente identificado segundo seu tipo, ou seja, primeiramente é necessário saber o tipo do dado para depois fazer seu armazenamento adequado. **Armazenado o dado, ele pode ser utilizado e processado a qualquer momento**.

Regras para identificadores:

Um identificador pode ter até 32 caracteres de comprimento.

- ❖ O 1º caractere deve ser uma letra do alfabeto.

- ❖ Os demais caracteres podem ser letras, números ou sinal *underscore* (`_`)

- ❖ Não usar sinais de pontuação, caracteres acentuados, cedilha (ç) ou espaço em branco.

- ❖ Exemplos de identificadores: **Nome, Nome_Cliente, Nome_Completo, Data, Data_Emissao, Mensagem, Aviso, 1aSemana, Endereco, a, b, c, ..., X, Y, Z;**

01.2 – Tipos de Dados na ling. C++ para ARDUINO

[bool](#)

[boolean](#)

[byte](#)

[char](#)

[double](#)

[float](#)

[int](#)

[long](#)

[short](#)

[size_t](#)

[string](#)

[String\(\)](#)

[unsigned char](#)

[unsigned int](#)

[unsigned long](#)

[vetor](#)

[void](#)

[word](#)

Uma variável representa um **contêiner ou espaço na memória** física ou virtual de um computador, **onde diferentes tipos de dados (valores) são armazenados durante a execução de um programa**. A cada variável é atribuído um nome descritivo ou identificador que se refere ao valor salvo.

A seguir alguns tipos de variáveis usadas na linguagem C++ para sistemas embarcados:

int armazena inteiros (números inteiros), sem decimais, como 123 ou -123

double armazena números de ponto flutuante, com decimais, como 19,99 ou -19,99

char armazena caracteres únicos, como 'a' ou 'B'. Os valores de caracteres estão entre aspas simples

string armazena texto, como "Olá Mundo". Os valores da string estão entre aspas duplas

bool armazena valores com dois estados: verdadeiro ou falso

01.2) TIPOS DE DADOS na ling. C++ do ARDUINO

TIPO DE DADO	CONSUMO DE MEMÓRIA (Bytes)	INTERVALO DE DADOS	OBSERVAÇÕES
boolean	1	Verdadeiro (True = 1) ou Falso (False = 0)	
char	1	-128 até +128	Usado para representar código de caractere ASCII (Ex.: Caractere A = 65)
byte	1	0 até 255	
int	2	-32.768 até +32.767	
unsigned int	2	0 até 65.536	Usado para ter precisão extra ao tipo int quando não há n ^{os} negativos
long	4	-2147483648, ..., -1, 0, 1, ..., 2147483647	Quando necessita-se representar números muito grandes
unsigned long	4	0 até 4294967295	
float	4	-3.4028235E+38 até +3.4028235E+38	Números com 7 dígitos significativos, Emáx=38
double	4	-3.4028235E+38 até +3.4028235E+38	Excepcionalmente no Arduino, ao contrario do que ocorre na linguagem C++ padrão, o tipo double terá comportamento idêntico ao float
String()	Cria uma instância String ¹ a partir de diferentes tipos de dados; esta pode ser usada para armazenar texto/caracteres e pode ser útil para delimitar casas decimais no tipo float.		

¹ Veja item 1.5 – Instância String na página 15 deste livro.

2- Operadores Aritméticos

são símbolos especiais que realizam as operações aritméticas básicas

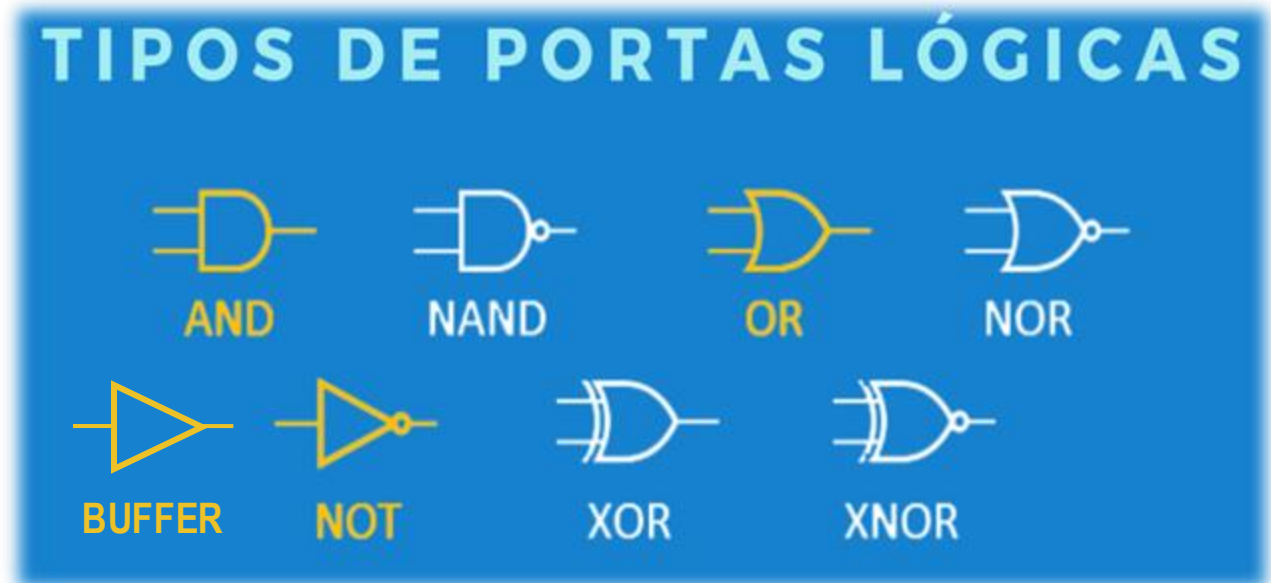
Operadores Aritméticos

OPERAÇÃO	SÍMBOLO	CÓDIGO EM C++	DESCRIÇÃO
ADIÇÃO	+	$x + y$	Efetua a soma $x + y$
SUBTRAÇÃO	-	$x - y$	Efetua a subtração $x - y$
MULTIPLICAÇÃO	*	$x * y$	Efetua a multiplicação $x * y$
DIVISÃO	/	x / y	Efetua a divisão x / y
RESTO DA DIVISÃO	%	$x \% y$	Obtém o resto da divisão entre dois inteiros
INCREMENTO	++	+=	Incrementa o valor da variável em 1 unidade
DECREMENTO	--	-=	Decrementa o valor da variável em 1 unidade

3 – Portas Lógicas e os Microcontroladores

Conforme apresentado neste capítulo, existem basicamente sete diferentes tipos de Portas Lógicas nos sistemas eletrônicos (**AND, OR, BUFFER, NOT, NAND, NOR, XOR, XNOR**).

Contudo, para seu uso na programação de microcontroladores, é necessário pensar apenas em quatro tipos principais (**AND, OR, BUFFER*, NOT**).]Observa-se que o Buffer já é nativo nos pinos Digitais e/ou Pinos de Propósito Geral (do inglês **General Purpose Input/Output - GPIO**), que permitem interação entre o microcontrolador e o mundo externo; eles garantem uma isolamento de impedância ao serem acoplados a sensores e/ou circuitos externos. Mas recomenda-se prudência e cautela ao se tratar com projetos mais complexos envolvendo acoplamento entre sensores e/ou circuitos de diferentes valores de impedância de saída e de entrada.



Linguagem C++	OPERADOR	SÍMBOLO	SAÍDA
	AND	&&	Retorna 1 apenas quando as duas entradas forem 1
	OR		Retorna 1 quando qualquer entrada for 1
	NOT	!	Retorna sempre o estado contrário do estado atual

Se a Linguagem de Programação C++ só tem códigos para as portas E, OU e NÃO, Como faço para obter as demais?

3- Portas Lógicas

com uControladores: Exemplo 1

//3.16 - Portas Lógicas com Microcontroladores (Exemplo 01)

int LED = 11; //define LED no Pino número 11

int botao1 = 0; //define botao 1

int botao2 = 1; //define botão 2

int estadobotao1 = 0; //define estadobotao1 como número 0

int estadobotao2 = 0; //define estadobotao2 como número 0

void setup() { // função de configuração de entrada e saída

// inicializa os pinos digitais como saídas.

pinMode(LED, OUTPUT); //configura o ledPin como SAÍDA

pinMode(botao1, INPUT); //configura o botão como ENTRADA

pinMode(botao2, INPUT);

}

void loop() //função de repetição

{

estadobotao1 = digitalRead(botao1); //armazena o nível lógico do botão

estadobotao2 = digitalRead(botao2);

if (estadobotao1 == 1 && estadobotao2 == 1) //Retoma 1 se Botão 1 em nível 1 E Botão 2 em nível 1

{

digitalWrite(LED, HIGH); //escreve na saída de nível alto(HIGH) igual a 5V

}

else // senão

{

digitalWrite(LED, LOW);

}

}

Por conseguinte, neste capítulo discutiremos a utilização da estrutura condicional if fazendo o uso desses operadores lógicos.

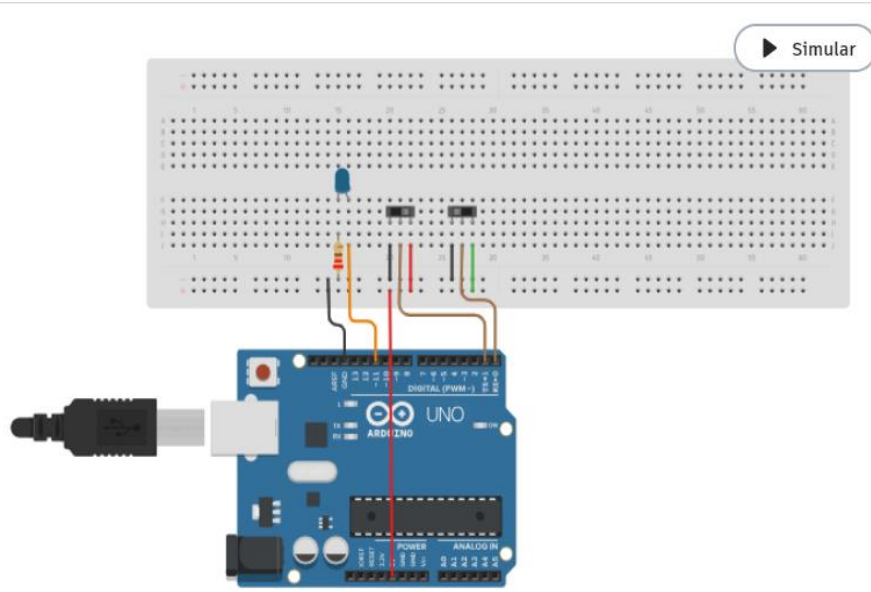
3.16 - Portas Lógicas com Microcontroladores (Exemplo 01)

Reagir 0

Circuito remixado por **Massaki Igarashi1**

[Editar isto](#)


SCAN ME



Na programação de microcontroladores as portas lógicas podem auxiliar na redução de linhas de códigos. Uma vez que vêm associados a uma sequência de comandos de estrutura de controle de fluxo usando estrutura condicional if; que por sinal, possibilitam diferentes possibilidades de algoritmos na lógica de programação.

3 – Portas Lógicas

com uControladores: Exemplo 2

//3.16 - Portas Lógicas com Microcontroladores (Exemplo 01)

```
int LED = 11; //define LED no Pino número 11
```

```
int botao1 = 0; //define botao 1
```

```
int botao2 = 1; //define botão 2
```

```
int estadobotao1 = 0; //define estadobotao1 como número 0
```

```
int estadobotao2 = 0; //define estadobotao2 como número 0
```

```
void setup() { // função de configuração de entrada e saída
```

```
  // inicializa os pinos digitais como saídas.
```

```
  pinMode(LED, OUTPUT); //configura o ledPin como SAÍDA
```

```
  pinMode(botao1, INPUT); //configura o botão como ENTRADA
```

```
  pinMode(botao2, INPUT);
```

```
}
```

```
void loop() //função de repetição
```

```
{
```

```
  estadobotao1 = digitalRead(botao1); //armazena o nível lógico do botão
```

```
  estadobotao2 = digitalRead(botao2);
```

```
  if ( estadobotao1 == 1 || estadobotao2 == 1) //Retoma 1 se Botão 1 em nível 1 OU Botão 2 em nível 1
```

```
  {
```

```
    digitalWrite(LED, HIGH); //escreve na saída de nível alto(HIGH) igual a 5V
```

```
  }
```

```
  else // senão
```

```
  {
```

```
    digitalWrite(LED, LOW);
```

```
  }
```

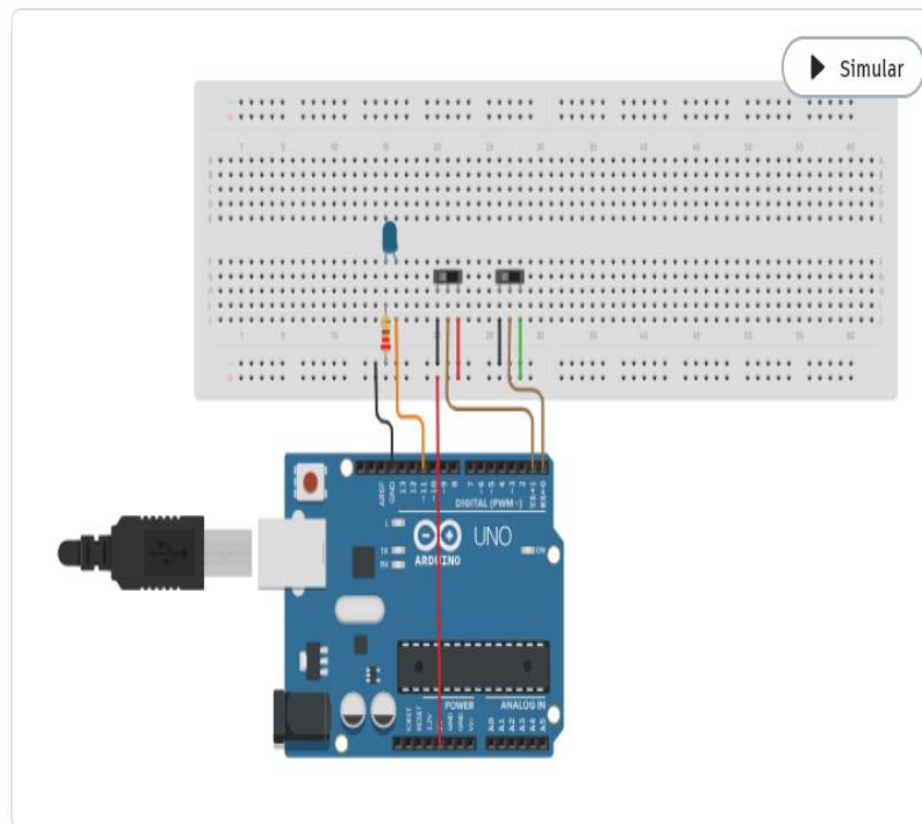
```
}
```

Por conseguinte, neste capítulo discutiremos a utilização da estrutura condicional if fazendo o uso desses operadores lógicos.



3.17 - Portas Lógicas com Microcontroladores (Exemplo 02)

Reagir 0



▶ Simular



Circuito remixado por
Massaki Igarashi1

Editar isto



Compartilhar na sala de aula

Copiar link



SCAN ME

3 – Portas Lógicas

com uControladores: Exemplo 3

//3.16 - Portas Lógicas com Microcontroladores (Exemplo 01)

int LED = 11; //define LED no Pino número 11

int botao1 = 0; //define botao 1

int botao2 = 1; //define botão 2

int estadobotao1 = 0; //define estadobotao1 como número 0

int estadobotao2 = 0; //define estadobotao2 como número 0

void setup() { // função de configuração de entrada e saída

// inicializa os pinos digitais como saídas.

pinMode(LED, OUTPUT); //configura o ledPin como SAÍDA

pinMode(botao1, INPUT); //configura o botão como ENTRADA

pinMode(botao2, INPUT);

}

void loop() //função de repetição

{

estadobotao1 = digitalRead(botao1); //armazena o nível lógico do botão

estadobotao2 = digitalRead(botao2);

if (estadobotao1 == 1 || estadobotao2 != 1) //Retoma 1 se Botão 1 em nível 1 OU Botão 2 em nível 0

{

digitalWrite(LED, HIGH); //escreve na saída de nível alto(HIGH) igual a 5V

}

else // senão

{

digitalWrite(LED, LOW);

}

}

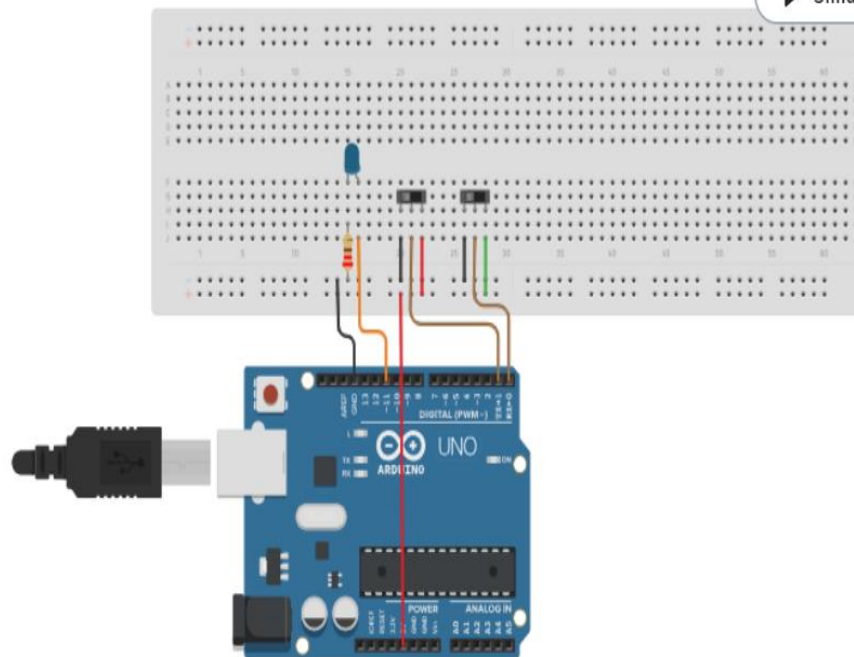
Por conseguinte, neste capítulo discutiremos a utilização da estrutura condicional if fazendo o uso desses operadores lógicos.



3.18 - Portas Lógicas com Microcontroladores (Exemplo 03)

Reagir 0

▶ Simular



Circuito remixado por
Massaki Igarashi1

Editar isto



Compartilhar na sala de aula



SCAN ME

03.19 – Portas Lógicas

com uControladores: Exemplo 4

```
//sensor de temperatura usando o LM35
#define ledRed 13
#define ledYellow 12
#define ledGreen 11
const int LM35 = A0; //define o pino que lera a saída do LM35
int temperatura; //variavel que armazenara a temperatura medida
void setup(){
  pinMode(ledRed, OUTPUT);
  pinMode(ledYellow, OUTPUT);
  pinMode(ledGreen, OUTPUT);
  Serial.begin(9600); //inicializa a comunicacao serial
}
void loop(){
  temperatura = float(analogRead(LM35))*5/(1024.0)/0.01;
  Serial.println(temperatura);
  delay(500);

  if(temperatura<25){
    digitalWrite(ledYellow, LOW);
    digitalWrite(ledRed, LOW);
    digitalWrite(ledGreen, HIGH);
  } else if (temperatura >= 25 && temperatura <= 30){
    digitalWrite(ledYellow, HIGH);
    digitalWrite(ledRed, LOW);
    digitalWrite(ledGreen, LOW);
  } else{ //Verifica se temperatura>30
    digitalWrite(ledYellow, LOW);
    digitalWrite(ledRed, HIGH);
    digitalWrite(ledGreen, LOW);
  }
}
```



03.19 - Portas Lógicas com uControladores (Exemplo 04)

Reagir 0

▶ Simular

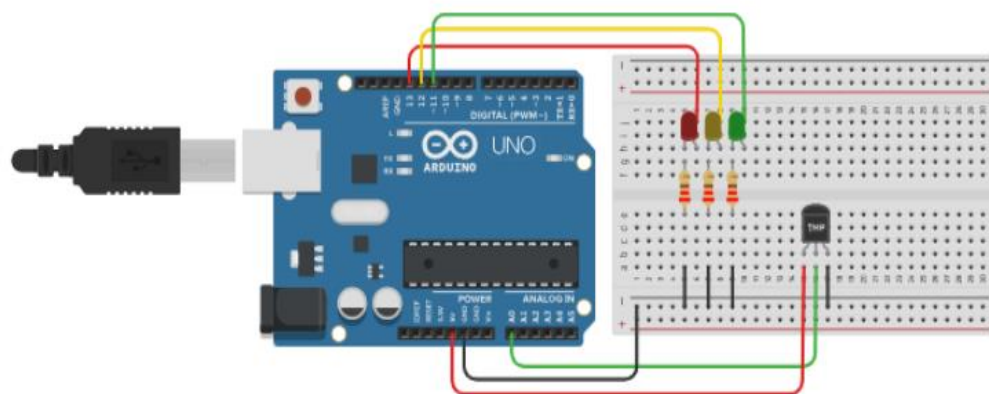


Circuito remixado por
Massaki Igarashi1

Editar isto



Compartilhar na sala de aula



SCAN ME

Atividade

Se em linguagem de programação C++ para microcontrolador Arduino só há disponível códigos para E, OU e NÃO. Você precisa pensar e responder como fazer para implementar as demais lógicas (NAND, NOR, e XOR); respondendo a pergunta: é possível implementá-las? E caso afirmativo, então como fazer para obtê-las mostrando um exemplo de programa para cada uma delas.